



UNIVERSIDAD AUTÓNOMA DE CHIHUAHUA

FACULTAD DE INGENIERÍA

CÓMPUTO PARALELO Y DISTRIBUIDO

INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

Actividad 13: Solución de problemas mediante
procesamiento paralelo (Asyncio, MPI y NVIDIA CUDA)

GRUPO:8CC2

JOSE SAUL DE LIRA MIRAMONTES

Emiliano Piñon Marín 367860

Luisa Fernanda Hernández Hernández 368068

Andrés Gonzales Alonso 367600

Mauricio Elías Navarrete Flores 367785

Chihuahua, Chih. 11/05/26

1. Descripción General

Este proyecto implementa un sistema de análisis predictivo de retrasos de vuelos utilizando técnicas de cómputo paralelo y distribuido.

La solución integra:

- Recolección concurrente de datos de vuelos y clima mediante AirLabs API y OpenWeatherMap API.
- Procesamiento paralelo con asyncio y Numba.
- Distribución de datos con MPI (mpi4py).
- Entrenamiento de un modelo de Machine Learning (CatBoostClassifier).
- Visualización interactiva en un dashboard web con Dash y Plotly

2. Arquitectura del Sistema

El flujo general del sistema se compone de cuatro módulos principales:

- **Recolección de datos**
 - Implementado en main.py.
 - Uso de asyncio y aiohttp para realizar peticiones concurrentes a las APIs.
 - Control de concurrencia con semáforo (asyncio.Semaphore(10)).
 - Almacenamiento de los datos en archivos CSV.
- **Procesamiento distribuido MPI**
 - Implementado en data_engine_mpi.py.
 - División del dataset en fragmentos con scatter.
 - Cálculo paralelo de métricas locales (retraso promedio, máximo, vuelos retrasados).
 - Consolidación de resultados con gather y reducción en el nodo maestro.
 - Generación de archivo cacheado mpi_cache_retrasos.csv.
- **Entrenamiento del modelo**
 - Implementado en modelotarea1.py.
 - Preprocesamiento de variables temporales y climáticas.
 - Ingeniería de características paralelizada con Numba.

- Optimización de hiperparámetros con RandomizedSearchCV y StratifiedKFold.
- Evaluación con métricas: Accuracy, Precision, Recall, F1, ROC AUC.
- Interpretabilidad con SHAP values e importancia de variables.
- Almacenamiento del modelo entrenado en models/modelotarea1.cbm.
- **Dashboard interactivo**
 - Implementado en appdash.py.
 - Visualización de métricas globales y resultados de MPI.
 - Gráficas de importancia de variables y distribución de retrasos.
 - Simulador de predicción de retrasos en tiempo real.
 - Interfaz moderna con CSS personalizado y componentes de Bootstrap.

3. Tecnologías Utilizadas

Tecnología	Propósito
Python	Lenguaje principal
asyncio / aiohttp	Concurrencia y peticiones asíncronas
Numba	Paralelización de funciones numéricas
mpi4py	Procesamiento distribuido con MPI
CatBoost	Modelo de clasificación
scikit-learn	Optimización y métricas
SHAP	Interpretabilidad del modelo
Dash + Plotly + Bootstrap	Dashboard interactivo
AirLabs API	Datos de vuelos
OpenWeatherMap API	Datos meteorológicos

4. Funcionalidades del Sistema

4.1 Recolección masiva de datos de vuelos y clima

El sistema obtiene información en tiempo real de vuelos y condiciones meteorológicas:

- Se implementa en main.py con asyncio y aiohttp, permitiendo realizar múltiples solicitudes concurrentes sin bloquear el flujo de ejecución.
- Se controla la concurrencia con un semáforo (asyncio.Semaphore(10)), evitando saturar las APIs externas.
- Los datos de vuelos provienen de AirLabs API (horarios, estado, retrasos, aeropuertos).
- Los datos meteorológicos provienen de OpenWeatherMap API (temperatura, humedad, visibilidad, viento, lluvia, nubosidad).
- Los resultados se consolidan en un archivo CSV, con nombre dinámico basado en la marca de tiempo.

Impacto: Esta funcionalidad garantiza un dataset robusto y actualizado, integrando variables aeronáuticas y climáticas para alimentar el modelo predictivo.

4.2 Procesamiento paralelo y distribuido con MPI

El módulo data_engine_mpi.py divide el dataset en fragmentos y los procesa en paralelo:

- Scatter: El nodo raíz reparte partes del dataset entre los procesos disponibles.
- Procesamiento local: Cada nodo calcula métricas de retraso (promedio, máximo, número de vuelos retrasados por aeropuerto).
- Gather: Los resultados parciales se envían de vuelta al nodo maestro.
- Reducción: El maestro unifica y consolida los resultados en un archivo cacheado (mpi_cache_retrasos.csv).

Impacto: Este enfoque reduce significativamente el tiempo de cálculo y permite escalar el análisis a grandes volúmenes de datos, optimizando la visualización en el dashboard.

4.3 Entrenamiento y evaluación del modelo predictivo

El módulo modelotarea1.py implementa el modelo de CatBoostClassifier:

- Preprocesamiento: Conversión de variables temporales y climáticas a formatos numéricos, creación de la variable binaria retrasado (>10 minutos).

- Ingeniería de características: Variables derivadas como hora pico, vuelo nocturno, delta de temperatura, clima severo y humedad alta. Estas funciones se paralelizan con Numba para mejorar rendimiento.
- Optimización: Se utiliza RandomizedSearchCV con validación cruzada estratificada (StratifiedKFold) para ajustar hiperparámetros (profundidad, learning rate, regularización, bagging).
- Evaluación: Se calculan métricas de clasificación (Accuracy, Precision, Recall, F1, ROC AUC) y se generan reportes como matriz de confusión y clasificación por clase.
- Interpretabilidad: Se emplean SHAP values y análisis de importancia de variables para entender el impacto de cada característica en las predicciones.
- Almacenamiento: El modelo entrenado se guarda en models/modelotarea1.cbm para su uso en el dashboard.

Impacto: El modelo permite anticipar retrasos con alta precisión, integrando factores operacionales y meteorológicos.

4.4 Dashboard interactivo

El módulo appdash.py ofrece una interfaz basada en nuestros resultados y otras peticiones a la api:

- Visualización de métricas globales: Retraso promedio, retraso máximo, número de vuelos analizados, número de aeropuertos procesados.
- Top 15 aeropuertos con mayor retraso: Gráfica alimentada por resultados de MPI.
- Importancia de variables: Gráficas horizontales que muestran las características más relevantes del modelo.
- Simulador de predicción: Permite ingresar datos de vuelo y condiciones climáticas para calcular la probabilidad de retraso en tiempo real.
- Indicadores visuales dinámicos: Gauge chart que muestra la probabilidad de retraso con colores de alerta (verde para puntualidad, rojo para riesgo).
- Alertas de riesgo operacional: Mensajes dinámicos que advierten sobre condiciones desfavorables.

Impacto: El dashboard convierte los resultados técnicos en información accesible y visual, útil para análisis operacional y toma de decisiones.

5. Ejecución del proyecto

1. Recolección de datos:

- a. Se ejecuta: `python3 main.py`

	flight_iata ▾	dep_iata ▾	dep_time ▾	dep_estimated ▾	arr_iata ▾	arr_time ▾	status ▾	delayed ▾	weather_temp ▾
1	E17403	ORD	2026-05-09 20:54	2026-05-10 00:04	MKE	2026-05-09 21:59	Landed	166	9.2
2	AA5001	ORD	2026-05-09 20:54	2026-05-10 00:04	MKE	2026-05-09 21:59	Landed	166	9.2
3	005001	ORD	2026-05-09 20:54	2026-05-10 00:04	MKE	2026-05-09 21:59	Landed	166	9.2
4	QR2859	ORD	2026-05-09 20:54	2026-05-10 00:04	MKE	2026-05-09 21:59	Landed	166	9.2
5	BA5260	ORD	2026-05-09 20:54	2026-05-10 00:04	MKE	2026-05-09 21:59	Landed	166	9.2
6	AS6552	ORD	2026-05-09 20:54	2026-05-10 00:04	MKE	2026-05-09 21:59	Landed	166	9.2
7	AS6411	ORD	2026-05-09 21:39	2026-05-10 00:07	TVC	2026-05-09 23:57	Landed	150	9.2
8	IB8349	ORD	2026-05-09 21:39	2026-05-10 00:07	TVC	2026-05-09 23:57	Landed	150	9.2
9	E17679	ORD	2026-05-09 21:39	2026-05-10 00:07	TVC	2026-05-09 23:57	Landed	150	9.2
10	YX4356	ORD	2026-05-09 21:39	2026-05-10 00:07	TVC	2026-05-09 23:57	Landed	150	9.2
11	QR9545	ORD	2026-05-09 21:39	2026-05-10 00:07	TVC	2026-05-09 23:57	Landed	150	9.2
12	BA5347	ORD	2026-05-09 21:39	2026-05-10 00:07	TVC	2026-05-09 23:57	Landed	150	9.2

2. Entrenamiento del modelo:

- a. Se ejecuta: `python3 modelotarea1.py`

```
C:\Users\Emi\Documents\PythonProject\.venv\Scripts\python.exe C:\Users\Emi\Documents\PythonProject\computo_paralelo\PROYECTO_2\modelotarea1.py
retrasado
0 8943
1 1057
Name: count, dtype: int64
Fitting 5 folds for each of 40 candidates, totalling 200 fits
[CV] END bagging_temperature=1, border_count=128, depth=8, iterations=600, l2_leaf_reg=9, learning_rate=0.01, random_strength=5; total time= 54.6s
[CV] END bagging_temperature=1, border_count=128, depth=8, iterations=600, l2_leaf_reg=9, learning_rate=0.01, random_strength=5; total time= 1.1min
[CV] END bagging_temperature=1, border_count=128, depth=8, iterations=600, l2_leaf_reg=9, learning_rate=0.01, random_strength=5; total time= 1.1min
[CV] END bagging_temperature=1, border_count=128, depth=8, iterations=600, l2_leaf_reg=9, learning_rate=0.01, random_strength=5; total time= 1.1min
[CV] END bagging_temperature=1, border_count=128, depth=8, iterations=600, l2_leaf_reg=9, learning_rate=0.01, random_strength=5; total time= 1.1min
[CV] END bagging_temperature=3, border_count=64, depth=8, iterations=600, l2_leaf_reg=1, learning_rate=0.1, random_strength=3; total time= 1.2min
[CV] END bagging_temperature=1, border_count=128, depth=6, iterations=800, l2_leaf_reg=5, learning_rate=0.05, random_strength=3; total time= 1.2min
[CV] END bagging_temperature=3, border_count=64, depth=8, iterations=600, l2_leaf_reg=1, learning_rate=0.1, random_strength=3; total time= 1.2min
[CV] END bagging_temperature=3, border_count=64, depth=8, iterations=600, l2_leaf_reg=1, learning_rate=0.1, random_strength=3; total time= 1.3min
[CV] END bagging_temperature=3, border_count=64, depth=8, iterations=600, l2_leaf_reg=1, learning_rate=0.1, random_strength=3; total time= 1.3min
[CV] END bagging_temperature=3, border_count=64, depth=8, iterations=600, l2_leaf_reg=1, learning_rate=0.1, random_strength=3; total time= 1.3min
A
```

3. Procesamiento paralelo con MPI:

- a. `mpiexec -n 4 python3 data_engine_mpi.py`

```
/home/blesky/PycharmProjects/computoparalelo/.venv/bin/python /home/blesky/PycharmProjects/computoparalelo/PROYECTO-2/appdash.py
[AeroBI] Caché MPI no encontrada. Ejecutando data_engine_mpi.py con 4 procesos...
[MPI_MASTER] Iniciando procesamiento con 4 nodos...
[MPI_MASTER] Dataset original cargado: 10000 filas.
[Nodo 1] Recibió 2500 filas para procesar.
[Nodo 2] Recibió 2500 filas para procesar.
[Nodo 3] Recibió 2500 filas para procesar.
[Nodo 0] Recibió 2500 filas para procesar.
[MPI_MASTER] Unificando resultados de todos los nodos...
[MPI_MASTER] Caché generado en data/mpi_cache_retrasos.csv
[MPI_MASTER] Tiempo de ejecución MPI: 0.0531 segundos
[AeroBI] data_engine_mpi.py completado exitosamente.
```

4. Dashboard interactivo:

a. python3 [appdash.py](#)

```
/home/blesky/PycharmProjects/computoparalelo/.venv/bin/python /home/blesky/PycharmProjects/computoparalelo/PROYECTO-2/appdash.py
[AeroBI] Caché MPI encontrada. Saltando ejecución de data_engine_mpi.py.
* Tip: There are .env files present. Install python-dotenv to use them.
Dash is running on http://127.0.0.1:8050/

* Serving Flask app 'appdash'
* Debug mode: on
[AeroBI] Caché MPI encontrada. Saltando ejecución de data_engine_mpi.py.
* Tip: There are .env files present. Install python-dotenv to use them.
```



A continuación presentamos el link del repositorio donde se puede encontrar el proyecto con más instrucciones sobre su funcionamiento:

<https://github.com/Mau1839/C-mputo-paralelo-2>